Solution: Valid Palindrome II

Problem Statement

Given string `s`, determine whether it's possible to make a given string palindrome by removing at most one character.

A palindrome is a word or phrase that reads the same backward as forward.

Examples

1. **Example 1**:
    - Input: `"racecar"`
    - Expected Output: `true`
    - Justification: The string is already a palindrome, so no removals are needed.
2. **Example 2**:
    - Input: `"abeccdeba"`
    - Expected Output: `true`
    - Justification: Removing the character 'd' forms the palindrome "abccba".
3. **Example 3**:
    - Input: `"abcdef"`
    - Expected Output: `false`
    - Justification: No single character removal will make this string a palindrome.

Solution

To solve this problem, we use a two-pointer approach that initiates at both ends of the string. These pointers move towards the center, comparing characters at each step. Upon encountering a mismatch, the algorithm decides whether to skip the character at the left or the right pointer. A helper function is used to check if the resulting substring (after skipping a character) forms a palindrome. This process is performed twice, once for each pointer. If either scenario results in a palindrome, the original string can be considered a valid palindrome after removing at most one character. This efficient method determines the feasibility of forming a palindrome with minimal alterations to the string.

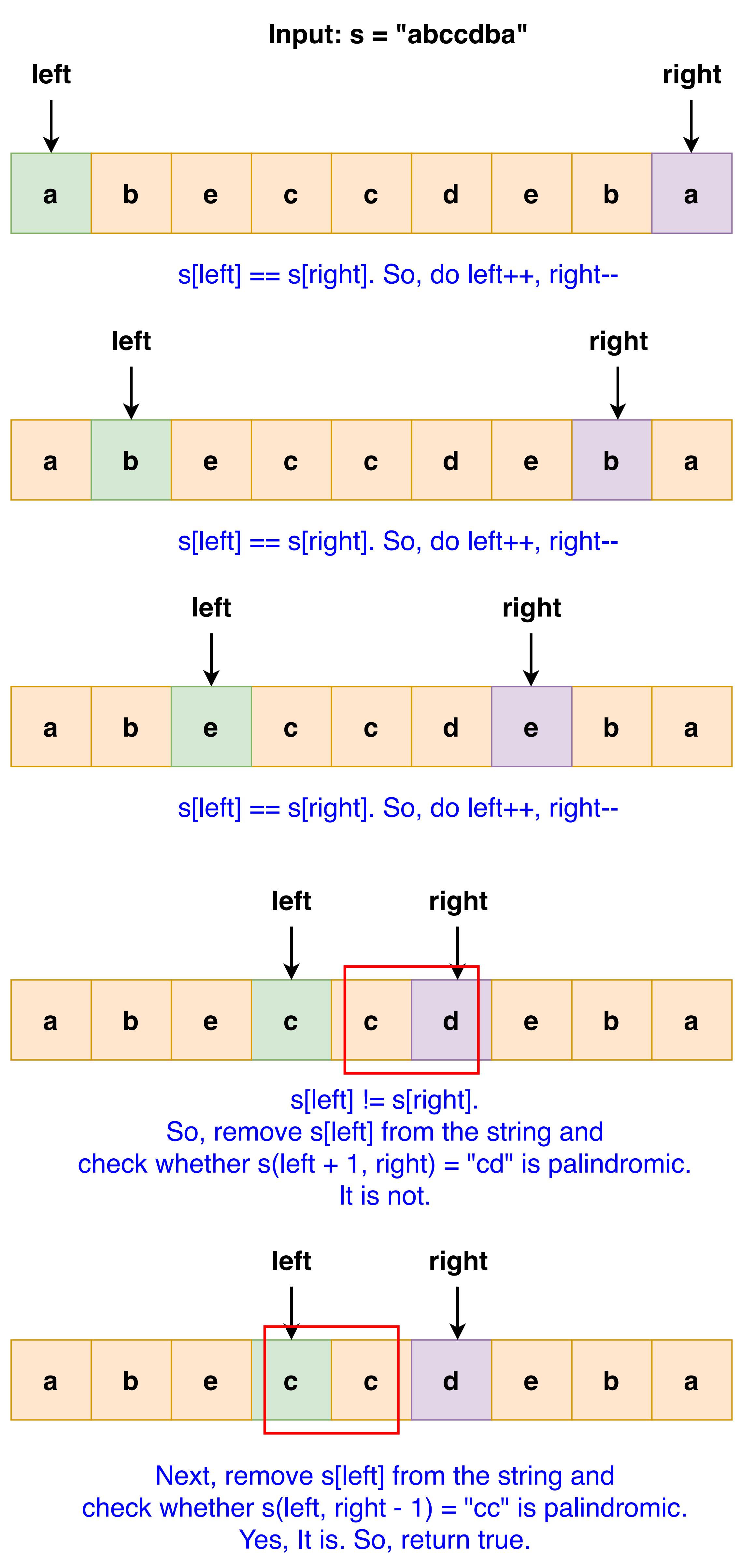
1. **Initialization**: Begin by initializing the `left` pointer with 0 and the `right` pointer with n - 1, where n is a string length.
2. **Two-Pointer Traversal**: Use two pointers, and move these pointers towards the center, comparing the characters at each step.
3. **Handling Mismatch**: Upon encountering a mismatch, the algorithm checks two scenarios: removing the character at the left pointer or at the right pointer. For each scenario, it checks if the remaining substring forms a palindrome.
4. **Greedy Decision Making**: If either resulting substring is a palindrome, return true. This decision is based on the greedy principle that choosing the first viable option (resulting in a palindrome) is sufficient.
5. **Concluding Result**: If neither scenario results in a palindrome, the algorithm concludes that it's impossible to form a palindrome by removing just one character and returns false.

This greedy approach is efficient as it minimizes the number of checks needed to determine if the string can be a valid palindrome with a single character removal.

Algorithm Walkthrough

Input:- "abeccdeba"

1. **Initial Setup**:
    - Two pointers are initialized: `left` at the start (pointing to `'a'`) and `right` at the end (pointing to `'a'`).
2. **First Iteration**:
    - Compare characters at `left` and `right` pointers.
    - Characters are the same (`'a'`), so move `left` to the right and `right` to the left.
3. **Second Iteration**:
    - Now `left` points to `'b'` and `right` points to `'b'`.
    - Characters match, so move `left` and `right` inward again.
4. **Third Iteration**:
    - `left` is now at `'e'`, and `right` is at `'e'`.
    - Characters match, so move `left` and `right` inward again.
5. **Fourth Iteration**:
    - `left` is now at `'c'`, and `right` is at `'d'`.
    - Characters do not match. This is where a decision is made.
6. **Checking Substrings**:
    - Remove the character at the `left` pointer (`'c'`) and check if the substring `"cd"` is a palindrome. It is not.
    - Remove the character at the `right` pointer (`'d'`) and check if the substring `"cc"` is a palindrome. It is a palindrome.
7. **Conclusion**:
    - Since removing the character `'d'` (at the `right` pointer) resulted in a palindrome, the answer is `true`.



s[left] == s[right]. So, do left++, right--

s[left] == s[right]. So, do left++, right--

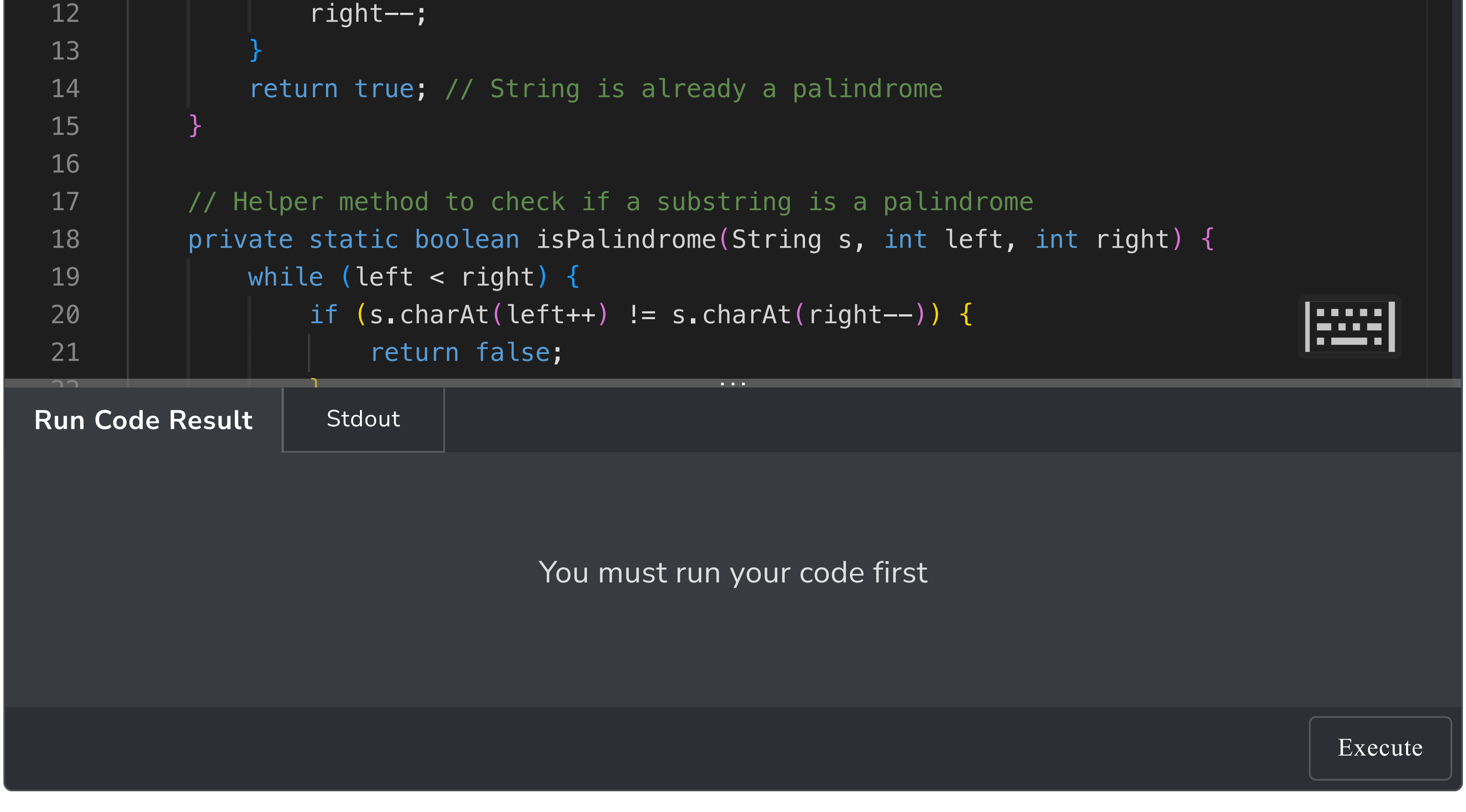
s[left] == s[right]. So, do left++, right--

s[left] != s[right].
So, remove s[left] from the string and
check whether s(left + 1, right) = "cd" is palindromic.
It is not.

Next, remove s[left] from the string and
check whether s(left, right - 1) = "cc" is palindromic.
Yes, It is. So, return true.

Code

Here is the code for this algorithm:



```
public class Solution {
    // Method to check if it's possible to form a palindrome
    public boolean isPalindromePossible(String s) {
        int left = 0, right = s.length() - 1;

        while (left < right) {
            if (s.charAt(left) != s.charAt(right)) {
                // Check by removing either left or right character
                return isPalindrome(s, left + 1, right) || isPalindrome(s, left, right -
            }
            left++;
            right--;
        }
        return true; // String is already a palindrome
    }

    // Helper method to check if a substring is a palindrome
    private static boolean isPalindrome(String s, int left, int right) {
        while (left < right) {
            if (s.charAt(left++) != s.charAt(right--)) {
                return false;
```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(n) for traversing the string.
- **Space Complexity**: O(1) as no extra space is used.

← Previous
Valid Palindrome II (Easy)

Next →
Maximum Length Of Pair Chain (Medium)

☐ Mark as Completed